

Write Sequence — Data Routing

FB15 ReadInverterData | PPC Energy | TIA Portal S7-1500

Implementation Reference & Race-Condition Checklist

1. ARCHITECTURE OVERVIEW

The SMA Sunny Central inverters are controlled over **Modbus TCP (FC16 write / FC03-FC04 read)**. All communication for a single inverter is handled by one instance of **FB15 ReadInverterData**, which owns the TCP connection and the *holdingRegisterMod* wire buffer. PPC logic and FB15 share data exclusively through the **SKID_DB.Inverter** structure (*Sunny_inverter* UDT). *holdingRegisterMod* is never written by PPC — it is only the Modbus byte-stream buffer.

Data flow for one OB30 tick (one inverter):

Step	Who runs	Action	Data location after this step
1	OB30 scheduler	PPC logic executes (PowerDistribution, ReactiveControl, FaultHandler)	SKID1.Inverter.InvOpMod = 308 SKID1.Inverter.RemRdy = 308 SKID1.Inverter."GriMng.WMod" = 1079 SKID1.Inverter.WSpt = <PPC kW> SKID1.Inverter.VArSpt = <PPC kVar> SKID1.Inverter.PFSpt = <PPC pf> SKID1.Inverter.ErrClr = 0 (or ack code)
2	FC19 READ_INVERTERS → FB15 (states 1-3)	Modbus reads: State 1: FC03 addr 0 len 104 State 2: FC04 addr 10 len 106 State 3: FC04 addr 116 len 112	holdingRegisterMod[] = raw Modbus words → decode FCs map measurements into: SKID1.Inverter.OpStt, InvMs.TotW, GriMs.Hz, WAval, VArAval, etc. (setpoint fields must NOT be overwritten — see Section 3)
3	FB15 FC_Pack_Write_Regs (states 4-6 transition)	Packs SKID DB setpoints into holdingRegisterMod[]	holdingRegisterMod[0..9] = ctrl group holdingRegisterMod[0..1] = WSpt holdingRegisterMod[0..3] = VArSpt+PFSpt
4	FB15 MB_CLIENT (states 4, 5, 6)	Modbus writes: State 4: FC16 addr 0 len 10 State 5: FC16 addr 108 len 2 State 6: FC16 addr 112 len 4	Physical inverter registers updated: reg 0-9: InvOpMod, RemRdy, VArMod, WMod, ErrClr reg 108-109: WSpt reg 112-115: VArSpt, PFSpt

2. FSM STATE SEQUENCE

FB15 uses a **FunctionalStateMachine (FSM_DB)** with **NumStates = 6**. The FSM advances on the rising edge of *clientData.done*. Each state fires one MB_CLIENT request. The full cycle takes 6 Modbus round-trips.

State	Direction	FC	Modbus addr	Words	Data mapped to
1 — READ	← Inverter	FC03 (mode 103)	0	104	Inverter (holding regs: InvOpMod, RemRdy, modes...) + ParamHold
2 — READ	← Inverter	FC04 (mode 104)	10	106	ParamInputs + Inverter measurements (OpStt, InvMs.TotW, GriMs.Hz, ...)
3 — READ	← Inverter	FC04 (mode 104)	116	112	ParamInputs + Inverter measurements (continued)
4 — WRITE	→ Inverter	FC16 (mode 1)	0	10	InvOpMod, RemRdy, GriMng.VArMod, GriMng.WMod, ErrClr
5 — WRITE	→ Inverter	FC16 (mode 1)	108	2	WSpt (active power setpoint)
6 — WRITE	→ Inverter	FC16 (mode 1)	112	4	VArSpt + PFSpt (reactive power + power factor)

Timing: Each state needs one Modbus TCP round-trip (~5–20 ms). Full 6-state cycle = ~30–120 ms per inverter. All 10 inverters run in parallel (each has its own MB_CLIENT IDB and TCP connection). OB30 = 100 ms.

3. RACE CONDITION — SETPOINT OVERWRITE RISK

■ States 1-3 (reads) run BEFORE states 4-6 (writes) within the same FSM cycle. If decode FCs map the FC04 readbacks of WSpt/VArSpt/PFSpt into SKID_DB.Inverter.WSpt / VArSpt / PFSpt, they will OVERWRITE the values PPC wrote in step 1 before MB_CLIENT has a chance to send them.

3.1 How to verify — check FC_InputReg10_L106_To_Inputs

State 2 reads FC04 input registers starting at address 10, length 106 words. This range covers addresses 10–115, which includes:

Modbus reg	SMA channel	Buffer offset in holdingRegisterMod	Risk field in Sunny_inverter UDT
108–109	WSpt readback	offset 196–197 ((108-10)×2)	Inverter.WSpt
112–113	VArSpt readback	offset 204–205 ((112-10)×2)	Inverter.VArSpt
114–115	PFSpt readback	offset 208–209 ((114-10)×2)	Inverter.PFSpt

Open FC_InputReg10_L106_To_Inputs in TIA Portal and locate the lines that read buffer positions 196–209. Check what UDT field they assign to:

What you find in FC_InputReg10_L106_To_Inputs	Status	Action required
Buffer[196..197] → #Inv.WSpt (Inverter UDT)	CONFLICT	Route to ParamInputs instead — add fields WSpt_Fdbk, VArSpt_Fdbk, PFSpt_Fdbk
Buffer[196..197] → #Inp.WSpt_Fdbk (ParamInputs UDT)	SAFE	No action needed — Inverter.WSpt stays as PPC commanded
Buffer[196..197] not mapped at all	SAFE	No action needed — readback discarded, Inverter.WSpt stays as PPC commanded

3.2 Same check for FC_Decode_Holding_0_103_V2 (State 1)

State 1 reads FC03 holding registers 0–103. The holding registers for InvOpMod (reg 0), RemRdy (reg 2), WMod (reg 6), VArMod (reg 4) return the values the inverter *last acknowledged* — which may lag PPC's command by one cycle. If the decode FC writes these back into Inverter.InvOpMod etc., PPC's new command for the same cycle will be overwritten.

Holding reg	Field	Buffer offset	Risk
0–1	InvOpMod	0–1	Inverter.InvOpMod overwritten with acknowledged (not commanded) value
2–3	RemRdy	2–3	Inverter.RemRdy overwritten
4–5	GriMng.VArMod	4–5	Inverter.VArMod overwritten
6–7	GriMng.WMod	6–7	Inverter.WMod overwritten
8–9	ErrClr	8–9	Inverter.ErrClr overwritten — one-shot lost

■ ErrClr is the most critical: it is a one-shot command. If State 1 reads back reg 8 (ErrClr = 0, because the inverter cleared it after executing) and the decode FC writes 0 back into Inverter.ErrClr, the one-shot is silently lost in the same cycle PPC set it.

Recommended fix if conflicts are found:

For the setpoint/command fields (InvOpMod, RemRdy, WMod, VArMod, ErrClr, WSpt, VArSpt, PFSpt): in the decode FCs, do **not** map the readback into the Inverter UDT field. Instead, store the readback in a separate *_Fdbk* field in ParamInputs or a dedicated feedback structure. This gives you both the PPC command AND the inverter-acknowledged value without conflict.

4. ERRCL ONE-SHOT IMPLEMENTATION

ErrClr must be written exactly **once per fault-clear request**. Writing it every cycle would keep sending the acknowledge command continuously, which the SMA inverter may interpret as repeated fault attempts.

Stage	Code / Network	Description
FaultHandler sets command	Set D1.Inverter.ErrClr := 302;	PPC writes acknowledge ENUM on rising edge of fault-clear request
State 4 transition: pack	FC_Pack_Write_Regs(WriteStep:=1, ...)	ErrClr value packed into holdingRegisterMod[8..9]
State 4: MB_CLIENT send	FC16 addr 0, len 10	ErrClr acknowledge sent to inverter
State 5 transition: self-clear	IF (FSM_DB.State=5) AND (FSM_DB.Transition) THEN #Inverter.ErrClr := 0;	State 5 starting = State 4 confirmed done. Clear ErrClr so next cycle writes 0 (no action).
Next cycle: State 4 send	holdingRegisterMod[8..9] = 0x00000000	Inverter receives ErrClr=0 — idle, no repeated acknowledge

✓ Use (FSM_DB.State = 5) AND (FSM_DB.Transition) to detect State 4 write completed. Write1_Done static variable is declared in FB15 but never driven — do not use it.

5. WRITE REGISTER MAP

Write State	FC16 Addr	Words	holdingRegister index	Mod field (source)	SMA channel	Format	Scaling
4	0	2	[0..1]	Inverter.InvOpMod	InvOpMod	S32 ENUM	1 (308=Run, 303=Stop)
4	2	2	[2..3]	Inverter.RemRdy	RemRdy	S32 ENUM	1 (308=Ready, 303=Standby)
4	4	2	[4..5]	Inverter."GriMng.VArMod"	GriMng.VArMod	S32 ENUM	1
4	6	2	[6..7]	Inverter."GriMng.WMod"	GriMng.WMod	S32 ENUM	1 (1079=WCtlCom)
4	8	2	[8..9]	Inverter.ErrClr	ErrClr	S32 ENUM	1 — ONE-SHOT
5	108	2	[0..1]	Inverter.WSpt	WSpt	S32 FIX0	×1, value in kW
6	112	2	[0..1]	Inverter.VArSpt	VArSpt	S32 FIX0	×1, value in kVAr
6	114	2	[2..3]	Inverter.PFSpt	PFSpt	S32 FIX4	Real×10000 e.g. 0.95→9500

6. IMPLEMENTATION CHECKLIST

#	Item to verify	Where to check	Pass condition
1	NumStates changed from 3 to 6 in FB15	Network 22 → FSM block NumStates input	NumStates value = 6
2	State 4/5/6 settings networks added to FB15	FB15 networks (after existing Network 31)	Only SCL networks: IF State=4/5/6 THEN set modbusMode=1, addr, len
3	FC_Pack_Write_Regs called on Transition for states 4, 5, 6	FB15 for states 4, 5, 6	Condition = (FSM_DB.State=N) AND (FSM_DB.Transition)
4	FC_Pack WriteStep 1: holdingRegisterMod[0..9] change with State 4	On Modbus channel write to State 4	holdingRegisterMod[0..9] for each Dint. e.g. InvOpMod=308 → [0]=0, [1]=3
5	FC_Pack WriteStep 2: holdingRegisterMod[0..9] change with State 5	On Modbus channel write to State 5	Value matches PPC commanded WSpt, not inverter readback
6	FC_Pack WriteStep 3: PFSpt scaling ×10000 match [2..3] during State 6	On Modbus channel write to State 6	PFSpt=0.9500 → Dint 9500 → [2]=0x0000, [3]=0x251C
7	FC_InputReg10_L106_To_Inputs does not FC16 source inverter WSpt/VArSpt/PFSpt	Open FC16 source inverter WSpt/VArSpt/PFSpt	WSpt/VArSpt/PFSpt map to ParamInputs (or unmapped), NOT Inverter UDT
8	FC_Decode_Holding_0_103_V2 does not FC16 source inverter WSpt/VArSpt/PFSpt	Open FC16 source inverter WSpt/VArSpt/PFSpt	WSpt/VArSpt/PFSpt fields not written by decode FC — only status/measurement fields u
9	ErrClr cleared after State 4 write completed	IF (State=5) AND (Transition) After FaultHandler.ErrClr	ErrClr goes 0 within one OB30 cycle after State 5 starts
10	No second MB_CLIENT instance on ParamInverterConnections table / Device	ParamInverterConnections table / Device	Each inverter IP has exactly one MB_CLIENT IDB (one per ReadInverterData
11	State 4 write: RemRdy written before FC16 Modbus mapping block	FC16 Modbus mapping block	RemRdy (FC16 Rdy) is physically after inverter processes atomically. If SMA re
12	ErrClr ENUM value correct for SMA	SMA Modbus register map for ErrClr (e.g. 308)	Confirm the ENUM code used by FaultHandler is the correct SMA acknowledge

Red rows = highest risk — must be verified before commissioning. Amber rows (in Section 3 tables) = verify in decode FC source.

7. FC17 RACE CONDITION — RESOLUTION

✓ Fix applied (Option B): FC_InputReg10_L106_To_Inputs (FC17) lines 174-175, 190-191, 194-195 redirected from Inv.WSpt/VArSpt/PFSpt to new fields Inps.WSpt_Fdbk/VArSpt_Fdbk/PFSpt_Fdbk.

FC17 line	Old assignment (CONFLICT)	New assignment (FIXED)
174–175	#Inv."WSpt" := #di32	#Inps."WSpt_Fdbk" := #di32
190–191	#Inv."VArSpt" := #di32	#Inps."VArSpt_Fdbk" := #di32
194–195	#Inv."PFSpt" := DINT_TO_REAL(#di32)/10000.0	#Inps."PFSpt_Fdbk" := DINT_TO_REAL(#di32)/10000.0

Three new fields must be added to **Skid_Parameters_Inputs** UDT: **WSpt_Fdbk : Dint, VArSpt_Fdbk : Dint, PFSpt_Fdbk : Real**. After this fix, Inv.WSpt / VArSpt / PFSpt are exclusively owned by PPC (command values). The inverter tracking confirmation is readable from Inps.*_Fdbk for HMI/SCADA comparison.

8. IEC_WATCHDOG — FB_PPC_CONTROLLER INPUT

IEC_Watchdog : Bool is the upstream SCADA / grid-operator communication alive signal wired directly to the **FB_PPC_Controller** input. When FALSE, the PPC controller stops following remote setpoints and enters safe mode (hold last setpoint or ramp to zero per plant policy). It is the software interlock that separates valid remote commands from stale or missing ones.

IEC_Watchdog	Meaning	PPC behaviour
TRUE	Upstream comms alive — setpoints valid and fresh	Normal dispatch — follow Targets_P / Q / PF from SCADA
FALSE	Upstream link dead or data stale — timeout expired	Safe mode — hold last setpoint or ramp to zero per plant policy. No new commands issued to inverters.

Wiring options by upstream protocol:

Upstream link	Wire IEC_Watchdog from
IEC 60870-5-104 (CP module, e.g. CP 1543-1)	CP_Block.Connected AND CP_Block.DataValid
IEC 60870-5-104 software stack	Stack connection status Bool + data-age check Bool
Modbus TCP from SCADA (PLC as slave)	Modbus server Connected output bit
Profinet IO or OPC UA from EMS	IO exchange quality bit / subscription active status
Commissioning — no upstream SCADA yet	Manual Bool bit e.g. %M300.0, set TRUE from HMI. NEVER hardwire TRUE in production.

Recommended implementation — retriggerable TON timer (protocol-independent):

```
// OB30 – SCADA_NewData = TRUE each time SCADA writes any new setpoint value
"IEC_WD_Timer"(IN := NOT "SCADA_NewData", PT := T#30S);
IEC_Watchdog := NOT "IEC_WD_Timer".Q;
// Timer.Q = TRUE → 30 s elapsed with no new data → IEC_Watchdog = FALSE
```

Why use a timer rather than just the TCP connection status: A Modbus TCP or IEC 104 connection can remain open (TCP keepalive active) while the SCADA application has crashed or stopped sending updates. The timer catches 'connected but silent' failures that the connection status bit alone cannot detect.

■ Never permanently hardwire IEC_Watchdog = TRUE in production. If the SCADA link fails with the watchdog forced TRUE, the PPC will continue dispatching the last received setpoints indefinitely with no alarm — a silent failure mode.

8.1 AuxCtl.LifeSign — SMA Inverter Application Watchdog

Separate from IEC_Watchdog, the SMA Sunny Central monitors a **lifesign counter** that the PLC must increment every cycle. If it stops changing for the configured SMA parameter **WtTms** (typically 60 s), the inverter drops out of remote control mode

and ignores Modbus setpoints — even if the TCP connection is still alive.

Item	Detail
SMA channel	AuxCtl.LifeSign
Direction	PLC → Inverter (FC16 write)
Data type	S16 (Int)
PLC action	Increment by 1 every OB30 cycle; wrap 32767 → 0
SMA timeout	WtTms parameter (default 60 s — verify in SMA service menu)
Readback	Already mapped in Skid_Parameters_Inputs.AuxCtl.LifeSign (from FC17)
Implementation	Add State 7 to FB15 FSM, OR include address in State 4 block if contiguous